ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

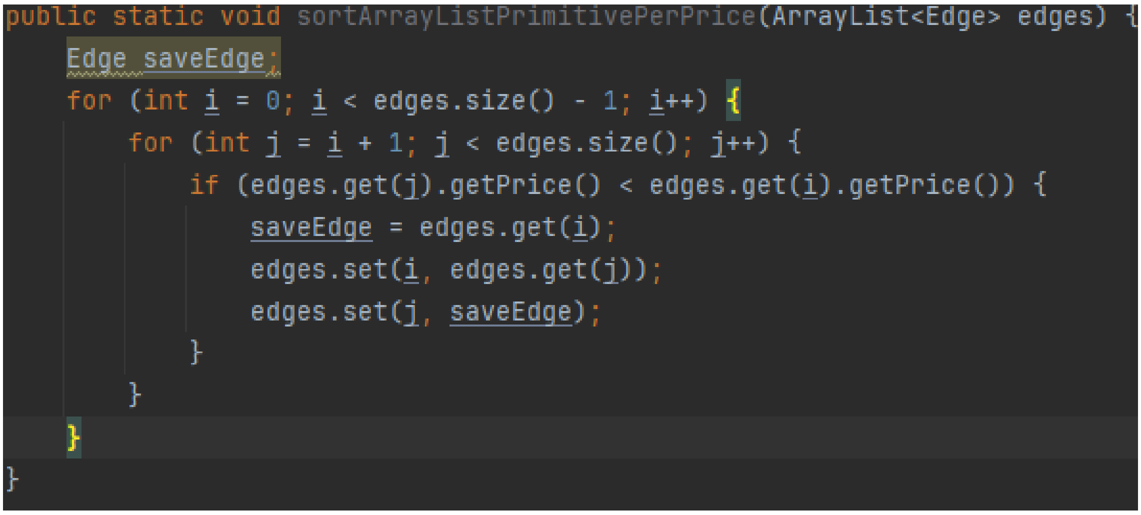
MDISC (Matemática Discreta)

Authors	Professors
Diogo Moreira - 1210527	AIM
Susana Loureiro - 1220804	ALG
Francisco Oliveira-1230684	ACD
Inês Oliveira-1231205	SAU

Subject
MDISC

Content

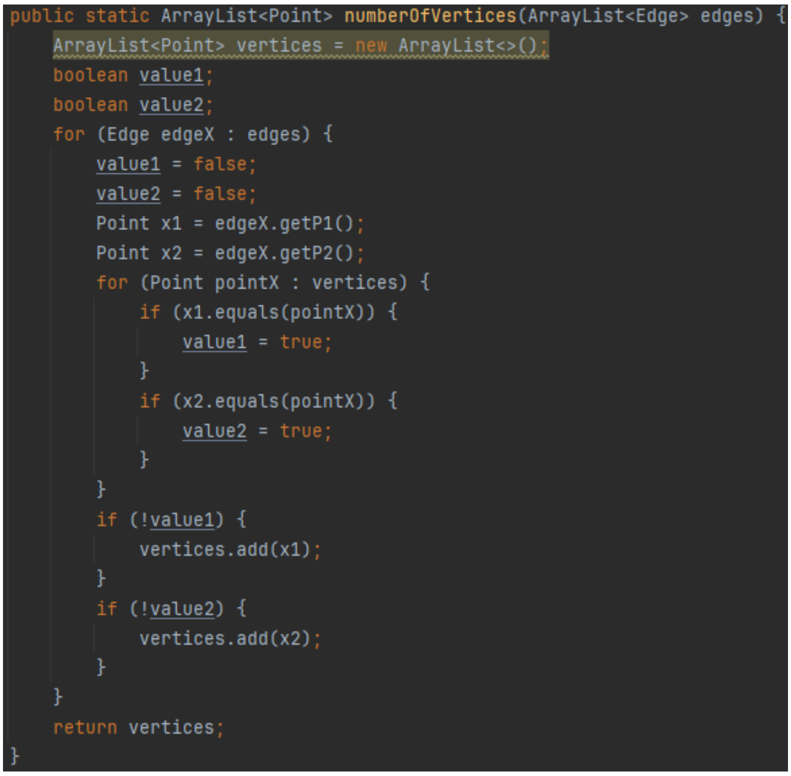
Explanation of procuderes



```
public static void sortArrayListPrimitivePerPrice(ArrayList<Edge> edges) {
    Edge saveEdge;
    for (int i = 0; i < edges.size() - 1; i++) {
        for (int j = i + 1; j < edges.size(); j++) {
            if (edges.get(j).getPrice() < edges.get(i).getPrice()) {
                saveEdge = edges.get(i);
                edges.set(i, edges.get(j));
                edges.set(j, saveEdge);
            }
        }
    }
}
```

FIGURA 1 - SORT ARRAY LIST PER PRICE

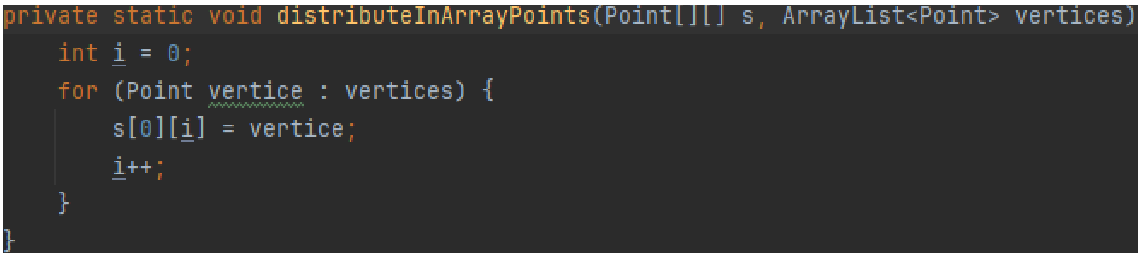
This method has the function of ordering the edges by cost.



```
public static ArrayList<Point> numberOfVertices(ArrayList<Edge> edges) {
    ArrayList<Point> vertices = new ArrayList<>();
    boolean value1;
    boolean value2;
    for (Edge edgeX : edges) {
        value1 = false;
        value2 = false;
        Point x1 = edgeX.getP1();
        Point x2 = edgeX.getP2();
        for (Point pointX : vertices) {
            if (x1.equals(pointX)) {
                value1 = true;
            }
            if (x2.equals(pointX)) {
                value2 = true;
            }
        }
        if (!value1) {
            vertices.add(x1);
        }
        if (!value2) {
            vertices.add(x2);
        }
    }
    return vertices;
}
```

FIGURA 2 - NUMBER OF VERTICES

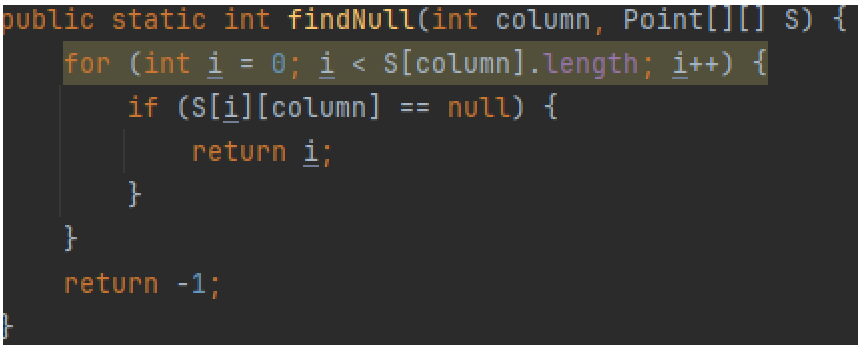
This method has the function to determine the vertices of the graph, and then determine where the Kruskal algorithm should stop.



```java
private static void distributeInArrayPoints(Point[][] s, ArrayList<Point> vertices)
    int i = 0;
    for (Point vertice : vertices) {
        s[0][i] = vertice;
        i++;
    }
}
```

Figura 3 - Distribute In Array Points

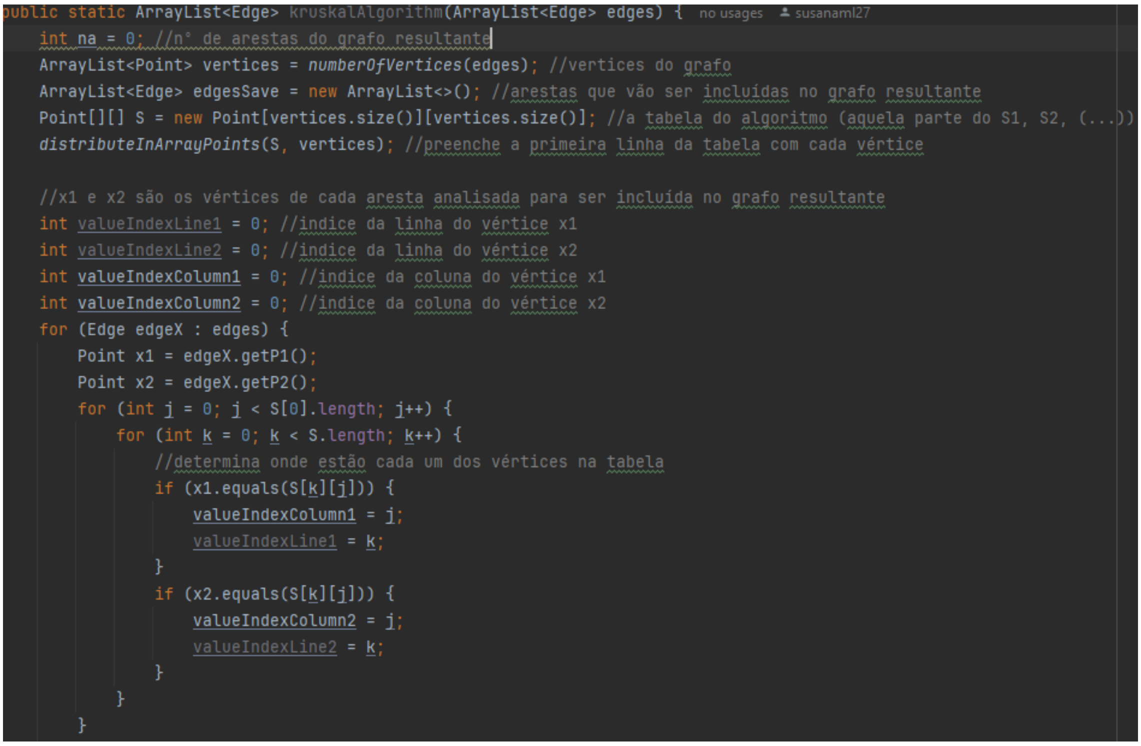
This method has the function to distribute the vértices across the first line.



```java
public static int findNull(int column, Point[][] S) {
    for (int i = 0; i < S[column].length; i++) {
        if (S[i][column] == null) {
            return i;
        }
    }
    return -1;
}
```

Figura 4 - Find Null

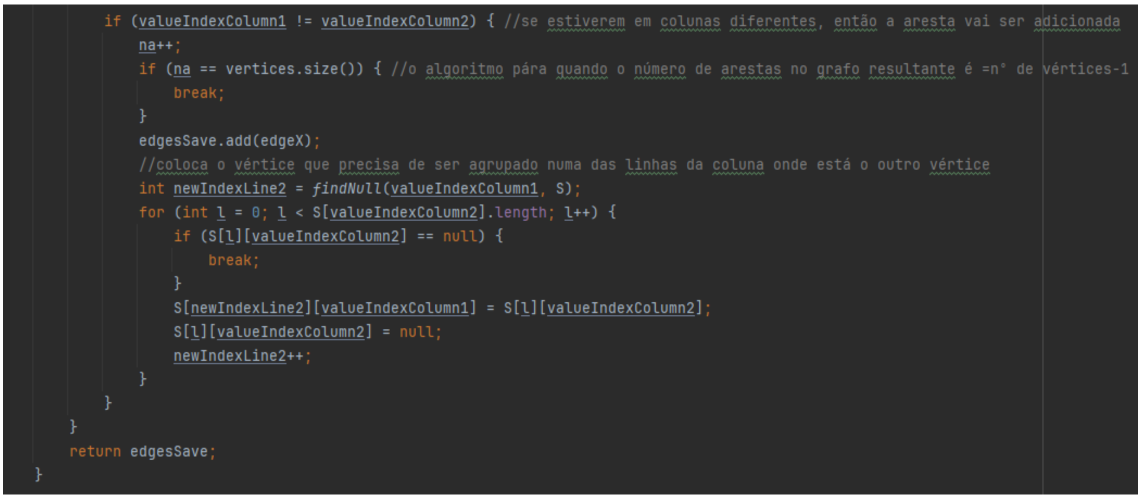
This method has the function to find a row or column that is empty to place the vertex to be grouped.



```java
public static ArrayList<Edge> kruskalAlgorithm(ArrayList<Edge> edges) {  no usages  susanaml27
    int na = 0; //n° de arestas do grafo resultante
    ArrayList<Point> vertices = numberOfVertices(edges); //vertices do grafo
    ArrayList<Edge> edgesSave = new ArrayList<>(); //arestas que vão ser incluídas no grafo resultante
    Point[][] S = new Point[vertices.size()][vertices.size()]; //a tabela do algoritmo (aquela parte do S1, S2, (...))
    distributeInArrayPoints(S, vertices); //preenche a primeira linha da tabela com cada vértice

    //x1 e x2 são os vértices de cada aresta analisada para ser incluída no grafo resultante
    int valueIndexLine1 = 0; //indice da linha do vértice x1
    int valueIndexLine2 = 0; //indice da linha do vértice x2
    int valueIndexColumn1 = 0; //indice da coluna do vértice x1
    int valueIndexColumn2 = 0; //indice da coluna do vértice x2
    for (Edge edgeX : edges) {
        Point x1 = edgeX.getP1();
        Point x2 = edgeX.getP2();
        for (int j = 0; j < S[0].length; j++) {
            for (int k = 0; k < S.length; k++) {
                //determina onde estão cada um dos vértices na tabela
                if (x1.equals(S[k][j])) {
                    valueIndexColumn1 = j;
                    valueIndexLine1 = k;
                }
                if (x2.equals(S[k][j])) {
                    valueIndexColumn2 = j;
                    valueIndexLine2 = k;
                }
            }
        }
```

Figura 5 - Kruskal Algorithm



```
            if (valueIndexColumn1 != valueIndexColumn2) { //se estiverem em colunas diferentes, então a aresta vai ser adicionada
                na++;
                if (na == vertices.size()) { //o algoritmo pára quando o número de arestas no grafo resultante é =n° de vértices-1
                    break;
                }
                edgesSave.add(edgeX);
                //coloca o vértice que precisa de ser agrupado numa das linhas da coluna onde está o outro vértice
                int newIndexLine2 = findNull(valueIndexColumn1, S);
                for (int l = 0; l < S[valueIndexColumn2].length; l++) {
                    if (S[l][valueIndexColumn2] == null) {
                        break;
                    }
                    S[newIndexLine2][valueIndexColumn1] = S[l][valueIndexColumn2];
                    S[l][valueIndexColumn2] = null;
                    newIndexLine2++;
                }
            }
        }
        return edgesSave;
    }
```

FIGURA 6 - KRUSKAL ALGORITHM CONTINUATION

This method is the Kruskal algorithm, with its explanation commented in the code. It was divided into two images for better understanding, as the algorithm is too large to place in a single image.